

Visual Programming

C# / WPF / ASP.NET — After Mid Complete Notes

COMSATS University Islamabad | Attock Campus

■ Table of Contents

- Lecture 13A — Threads & Multithreading
- Lecture 13B — ADO.NET (Database Connectivity)
- Lecture 15 — Tasks, async / await
- Lecture 18 — Data Binding in WPF
- Lecture 19 — LINQ (Language Integrated Query)
- Lecture 20 — Logical & Visual Tree, Dependency Properties
- Lecture 24 — Introduction to ASP.NET MVC
- Lecture 25 — ASP, ASP.NET, ASP.NET Core
- Lecture 26 — Razor View Engine
- Lecture 27 — LINQ to SQL
- Lecture 29 — Entity Framework
- Lecture 31 — Model Binding in ASP.NET MVC

Lecture 13A — Threads & Multithreading (C#)

■ Definition

A **Thread** is the smallest unit of execution within a process. Multithreading allows a program to run multiple threads concurrently, making applications faster and more responsive. Unlike processes, threads share the same memory space, so they are lightweight.

■ Why We Use It

- Keep the UI responsive while doing background work (e.g., file download).
- Perform heavy calculations without freezing the application.
- Distinguish tasks of varying priority.
- Utilize multi-core processors efficiently.

■ Syntax — Creating a Thread

```
// Basic Thread Creation
Thread thread = new Thread(new ThreadStart(MethodName));
thread.Name = "MyThread";
thread.Priority = ThreadPriority.Normal;    // optional
thread.Start();                             // start execution
thread.Join();                             // wait for thread to finish
```

■ Full Working Example — Thread Basics

[illegible]

```
}
```

■ Output

INPUT	OUTPUT
(none)	Thread: Worker-1 → 1 Thread: main → 1 Thread: Worker-1 → 2 Thread: main → 2 ... All threads done!

■ Thread States

A thread moves through these states during its lifetime:

State	Meaning
Unstarted	Thread object created but Start() not called yet.
Running	Thread is executing.
WaitSleepJoin	Thread called Sleep(), Wait(), or Join() — temporarily paused.
Blocked	Waiting for a lock or I/O operation.
Stopped	Thread completed or Abort() was called.

■ Thread Priority

```
Thread t = new Thread(new ThreadStart(DoWork));  
t.Priority = ThreadPriority.Highest;    // Lowest, BelowNormal, Normal, AboveNormal, Highest  
t.Start();
```

■ Thread Synchronisation — Monitor / lock

When multiple threads access shared data simultaneously, data corruption can occur. We use **lock** (syntactic sugar over Monitor.Enter/Exit) to allow only one thread at a time.

```
using System;  
using System.Threading;  
class BankAccount  
{  
    private decimal balance = 1000;  
    private readonly object _lock = new object();    // lock object  
    // Deposit method — thread-safe  
    public void Deposit(decimal amount)  
    {  
        lock (_lock)        // only ONE thread executes this block at a time  
        {  
            balance += amount;  
            Console.WriteLine($"Deposited {amount}. Balance = {balance}");  
        }  
    }  
    // Withdraw method — thread-safe
```

```

    public void Withdraw(decimal amount)
    {
        lock (_lock)
        {
            if (balance >= amount)
            {
                balance -= amount;
                Console.WriteLine($"Withdrew {amount}. Balance = {balance}");
            }
            else
                Console.WriteLine("Insufficient funds!");
        }
    }
}

class Program
{
    static BankAccount account = new BankAccount();
    static void Main()
    {
        // Two threads accessing the same account simultaneously
        Thread t1 = new Thread(() => account.Deposit(500));
        Thread t2 = new Thread(() => account.Withdraw(300));

        t1.Start(); t2.Start();
        t1.Join(); t2.Join();
        Console.WriteLine("Done!");
    }
}

```

■ Multithreading with GUI (WinForms) — Invoke

GUI controls can only be updated from the UI thread. Use **Control.Invoke()** to safely update a label/textbox from a background thread.

```

using System;
using System.Threading;
using System.Windows.Forms;

public partial class MainForm : Form
{
    private delegate void UpdateLabelDelegate(string text); // delegate for cross-thread calls
    private void UpdateLabel(string text)
    {
        // Must always update UI from UI thread
        if (label1.InvokeRequired)
            label1.Invoke(new UpdateLabelDelegate(UpdateLabel), text);
        else
            label1.Text = text;
    }

    private void DoBackgroundWork()
    {

```

```

        for (int i = 0; i <= 100; i++)
        {
            Thread.Sleep(50);
            UpdateLabel($"Progress: {i}%");    // safe cross-thread call
        }
    }

    private void btnStart_Click(object sender, EventArgs e)
    {
        Thread worker = new Thread(DoBackgroundWork);
        worker.IsBackground = true;    // thread dies when app closes
        worker.Start();
    }
}

```

■ Important Notes

- Always call `Join()` if the main thread needs the result of a worker thread.
- Use lock to protect shared resources — never access shared data without synchronization.
- `Thread.Sleep(ms)` pauses execution for milliseconds — does NOT block the CPU permanently.
- `IsBackground = true` means the thread will auto-stop when the main app closes.
- `Thread.Abort()` is deprecated in .NET 5+; use `CancellationToken` instead.

■ Real-World Use Cases

- File download in background while UI stays responsive.
- Web server handling multiple client requests simultaneously.
- Database queries running in parallel for faster results.
- Game engines updating physics, AI, and rendering on separate threads.

■ Interview Questions

Q: What is the difference between a Process and a Thread?

A: A process has its own memory space (heavyweight). Threads share memory within a process (lightweight). Switching between threads is faster than switching between processes.

Q: What is `Thread.Sleep()` and does it block the CPU?

A: `Thread.Sleep(ms)` pauses the current thread for given milliseconds. The thread enters `WaitSleepJoin` state. The CPU is NOT blocked — it runs other threads during this time.

Q: What is the purpose of lock keyword in C#?

A: lock ensures only one thread at a time can enter a critical section. It internally uses `Monitor.Enter()` and `Monitor.Exit()`. This prevents race conditions on shared data.

Q: What is a Race Condition?

A: When two threads access shared data simultaneously and the result depends on execution order. This causes unpredictable bugs. Solution: use lock/Monitor.

Q: What is `Thread.Join()`?

A: `Join()` makes the calling thread wait until the target thread finishes. Useful when main thread needs the result of a worker thread before proceeding.

Lecture 13B — ADO.NET — Database Connectivity

■ Definition

ADO.NET (ActiveX Data Objects .NET) is a set of classes in the .NET Framework that provides database access. It allows C# applications to connect to SQL Server, Oracle, MySQL etc., execute queries, and retrieve results.

■ Key ADO.NET Classes

Class	Purpose
SqlConnection	Opens/closes connection to SQL Server database.
SqlCommand	Executes SQL queries or stored procedures.
SqlDataReader	Reads data row-by-row (forward-only, fast, read-only).
SqlDataAdapter	Fills a DataSet from database; also updates database.
DataSet	In-memory cache of data — can hold multiple tables.
DataTable	A single table of in-memory data.

■ Connection String

```
// Format: Server=<server>;Database=<db>;User Id=<user>;Password=<pass>;
string connStr = "Server=localhost;Database=SchoolDB;User Id=sa;Password=1234;";
// For Windows Authentication (no password needed):
string connStr2 = "Server=.;Database=SchoolDB;Integrated Security=True;";
```

■ Full Example — SELECT with SqlDataReader

```
using System;
using System.Data.SqlClient;    // ADO.NET namespace
class ADOExample
{
    static void Main()
    {
        // ■■ Step 1: Define connection string ████████████████████████
        string connStr = "Server=localhost;Database=SchoolDB;" +
                        "User Id=sa>Password=1234;";

        // ■■ Step 2: Create Connection ████████████████████████████████
        using (SqlConnection conn = new SqlConnection(connStr))
        {
            conn.Open();    // open connection to database
            Console.WriteLine("Connected to Database!");

            // ■■ Step 3: Write SQL query ████████████████████████████████
            string query = "SELECT StudentID, Name, Marks FROM Students " +
                          "WHERE Marks > @minMarks";

            // ■■ Step 4: Create Command ████████████████████████████████
            using (SqlCommand cmd = new SqlCommand(query, conn))
```

```
{
    // Use parameters to prevent SQL Injection
    cmd.Parameters.AddWithValue("@minMarks", 50);

    // ■■ Step 5: Execute and read results ■■■■■■■■■■
    using (SqlDataReader reader = cmd.ExecuteReader())
    {
        while (reader.Read())    // read row by row
        {
            int id      = reader.GetInt32(0);        // column 0
            string name = reader.GetString(1);        // column 1
            float marks = reader.GetFloat(2);         // column 2
            Console.WriteLine($"ID:{id} Name:{name} Marks:{marks}");
        }
    }

    } // connection auto-closed here (using block)
}
```

■ INSERT / UPDATE / DELETE — ExecuteNonQuery

[illegible]

```
}
```

■ Output

INPUT	OUTPUT
(DB has 3 students with marks > 50)	Connected to Database! ID:1 Name:Ahmed Marks:75 ID:2 Name:Sara Marks:90 ID:3 Name:Usman Marks:65

■ Interview Questions

Q: What is the difference between SqlDataReader and DataSet?

A: SqlDataReader: forward-only, read-only, connected to DB (fast). DataSet: disconnected, stores full table in memory, supports navigation in any direction.

Q: What is ExecuteNonQuery() used for?

A: Used for INSERT, UPDATE, DELETE statements. Returns the number of rows affected. Does NOT return data.

Q: What is a Parameterized Query and why use it?

A: A query that uses @param placeholders instead of string concatenation. Prevents SQL Injection attacks and is safer for user input.

Q: What does using() block do in ADO.NET?

A: Ensures the connection/command is properly closed and disposed, even if an exception occurs. Same as try-finally with Dispose().

Lecture 15 — Tasks, async / await — Asynchronous Programming

■ Definition

Asynchronous programming lets your code start a long-running operation (e.g., file download, database query) and continue doing other work instead of blocking and waiting. In C#, this is done using **Task**, **async**, and **await** keywords.

■ Problem with Synchronous Code

```
// BAD – synchronous. UI freezes for 3 seconds!
private void btnDownload_Click(object sender, EventArgs e)
{
    Thread.Sleep(3000);    // simulates slow download – FREEZES UI
    label1.Text = "Downloaded!";
}
```

■ Solution — async / await

```
// GOOD – asynchronous. UI stays responsive!
private async void btnDownload_Click(object sender, EventArgs e)
{
    label1.Text = "Downloading...";
    await Task.Delay(3000);    // waits WITHOUT blocking the UI thread
    label1.Text = "Downloaded!";
}
```

■ Full Working Example — async/await

```
using System;
using System.Threading.Tasks;

class AsyncDemo
{
    // async method must return Task or Task<T>
    static async Task<int> CalculateSumAsync(int n)
    {
        Console.WriteLine("Calculation started...");
        await Task.Delay(2000);    // simulates 2-second delay (e.g., DB query)
        int sum = 0;
        for (int i = 1; i <= n; i++)
            sum += i;
        return sum;    // returned value is wrapped in Task<int>
    }

    static async Task Main()
    {
        Console.WriteLine("Main: before await");
        int result = await CalculateSumAsync(100);    // await = wait here until done
        Console.WriteLine($"Main: Sum = {result}");
        Console.WriteLine("Main: after await");
    }
}
```

```
}  
}
```

■ Running Multiple Tasks in Parallel

```
using System;  
using System.Threading.Tasks;  
class ParallelTasks  
{  
    static async Task<string> DownloadPage(string url)  
    {  
        await Task.Delay(1000); // simulate network delay  
        return $"Content from {url}";  
    }  
    static async Task Main()  
    {  
        // Start all 3 downloads AT THE SAME TIME  
        Task<string> t1 = DownloadPage("google.com");  
        Task<string> t2 = DownloadPage("github.com");  
        Task<string> t3 = DownloadPage("openai.com");  
        // Wait for ALL to complete (runs in parallel)  
        string[] results = await Task.WhenAll(t1, t2, t3);  
        foreach (var r in results)  
            Console.WriteLine(r);  
    }  
}
```

■ Output

INPUT	OUTPUT
(none)	Main: before await Calculation started... Main: Sum = 5050 Main: after await

■ Important Notes

- async method MUST contain at least one await, otherwise it runs synchronously.
- async void — only for event handlers. Use async Task for everything else.
- await does NOT create a new thread. It frees the current thread to do other work.
- Task.WhenAll() runs multiple tasks in parallel — faster than awaiting one-by-one.
- Task.WhenAny() returns as soon as the FIRST task completes.

■ Interview Questions

Q: What is the difference between Task and Thread?

A: Thread: OS-level, always creates a new thread (heavyweight). Task: higher-level abstraction over ThreadPool, more efficient, supports async/await pattern.

Q: Can we use await without async?

A: No. await can only be used inside a method marked with async keyword.

Q: What does async void mean?

A: async void is used for event handlers only. It cannot be awaited and exceptions are harder to handle. Prefer async Task.

Q: What is Task.WhenAll() vs Task.WhenAny()?

A: WhenAll: waits for ALL tasks to complete. WhenAny: returns as soon as ANY one task completes.

Lecture 18 — Data Binding in WPF

■ Definition

Data Binding in WPF is a mechanism that connects the UI controls (View) to data objects (Model) automatically. When the data changes, the UI updates automatically — and vice versa. This eliminates writing repetitive code to sync data.

■ Types of Binding Modes

Mode	Description
OneWay	Source → Target. UI updates when data changes. (e.g., display-only label)
TwoWay	Source ↔ Target. UI and data stay in sync. (e.g., TextBox editing)
OneTime	Source → Target only ONCE at startup.
OneWayToSource	Target → Source only. (rarely used)

■ Simple Binding — XAML

```
<!-- Binding TextBox.Text to a property called 'Name' -->
<TextBox Text="{Binding Path=Name, Mode=TwoWay}" />

<!-- Shorthand (Path= is optional) -->
<TextBox Text="{Binding Name}" />

<!-- Binding a slider to a TextBlock font size -->
<Slider Name="mySlider" Minimum="10" Maximum="40" Value="16"/>
<TextBlock Text="Hello WPF!"
           FontSize="{Binding ElementName=mySlider, Path=Value}"/>
```

■ C# Code-Behind — Setting DataContext

```
// Person.cs – Model class (must implement INotifyPropertyChanged for live updates)
using System.ComponentModel;

public class Person : INotifyPropertyChanged
{
    private string _name;
    private int _age;
    public string Name
    {
        get => _name;
        set
        {
            _name = value;
            OnPropertyChanged("Name");    // notify UI that Name changed
        }
    }
    public int Age
    {
        get => _age;
```

```

        set
        {
            _age = value;
            OnPropertyChanged("Age");
        }
    }

    public event PropertyChangedEventHandler PropertyChanged;
    protected void OnPropertyChanged(string propertyName)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }
}

// MainWindow.xaml.cs – Set DataContext in constructor
public partial class MainWindow : Window
{
    Person person = new Person { Name = "Ahmed", Age = 21 };

    public MainWindow()
    {
        InitializeComponent();
        this.DataContext = person;    // all bindings use 'person' as source
    }

    private void btnBirthday_Click(object sender, RoutedEventArgs e)
    {
        person.Age++;    // UI automatically updates because of INotifyPropertyChanged
    }
}

```

■ XAML — Complete Binding Example

```

<Window x:Class="BindingDemo.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Data Binding Demo" Height="200" Width="350">
    <StackPanel Margin="20">
        <TextBlock Text="Name:"/>
        <TextBox Text="{Binding Name, Mode=TwoWay, UpdateSourceTrigger=PropertyChanged}" />
        <TextBlock Text="Age:"/>
        <TextBox Text="{Binding Age, Mode=TwoWay}" />
        <Button Content="Happy Birthday!" Click="btnBirthday_Click" Margin="0,10,0,0" />
    </StackPanel>
</Window>

```

■ UpdateSourceTrigger Options

- **Default** — updates naturally based on control type (TextBox: LostFocus).
- **PropertyChanged** — updates source every keystroke (real-time).
- **LostFocus** — updates when user clicks away from control.
- **Explicit** — only when BindingExpression.UpdateSource() is called manually.

■ Binding to Another Control

```
<!-- Button foreground follows TextBox foreground -->
<TextBox Name="ageBox" Foreground="Red" Text="21"/>
<Button Content="Click Me"
        Foreground="{Binding ElementName=ageBox, Path=Foreground}"/>
```

■ Interview Questions

Q: What is `INotifyPropertyChanged` used for?

A: It's an interface that notifies the WPF binding engine when a property value changes, so the UI auto-updates. Without it, binding works only once.

Q: What is `DataContext` in WPF?

A: `DataContext` is an inherited property on every `FrameworkElement`. It sets the default data source for all bindings within that element and its children.

Q: What is the difference between `OneWay` and `TwoWay` binding?

A: `OneWay`: only source updates target (display only). `TwoWay`: source and target stay synchronized — changes in UI also update the model.

Lecture 19 — LINQ — Language Integrated Query

■ Definition

LINQ (Language Integrated Query) is a feature in C# that lets you query data from collections, databases, XML, and more using SQL-like syntax directly inside C# code. It provides compile-time checking, IntelliSense, and cleaner, more readable code compared to manual loops.

■ Two Syntax Styles

```
// ■■ 1. QUERY SYNTAX (looks like SQL)
var result = from n in numbers
              where n > 5
              orderby n
              select n;

// ■■ 2. METHOD SYNTAX (lambda expressions)
var result = numbers.Where(n => n > 5)
                    .OrderBy(n => n);
```

■ Full Working Example — LINQ on Collections

[illegible]

```
Console.WriteLine("First even: " + numbers.First(n => n % 2 == 0));  
Console.WriteLine("Last: " + numbers.Last());  
Console.WriteLine("Max: " + numbers.Max());  
Console.WriteLine("Min: " + numbers.Min());  
Console.WriteLine("Sum: " + numbers.Sum());  
Console.WriteLine("Count: " + numbers.Count());  
Console.WriteLine("Average: " + numbers.Average());  
  
// ■■ STRING filtering  
var longNames = names.Where(n => n.Length > 4).OrderBy(n => n);  
Console.WriteLine("Long names: " + string.Join(", ", longNames));  
  
// ■■ GROUPBY  
var grouped = names.GroupBy(n => n.Length);  
foreach (var group in grouped)  
{  
    Console.WriteLine($"Length {group.Key}: {string.Join(", ", group)}");  
}  
  
// ■■ CONTAINS / ANY / ALL  
Console.WriteLine("Contains 7: " + numbers.Contains(7));  
Console.WriteLine("Any > 9: " + numbers.Any(n => n > 9));  
Console.WriteLine("All > 0: " + numbers.All(n => n > 0));  
}
```

■ Output

INPUT	OUTPUT
(list defined in code)	Evens: 8, 2, 4, 6, 10 Sorted: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 Squares: 25, 9, 64, 1, 81, 4, 49, 16, 100 Max: 10 Min: 1 Sum: 55 Average: 5.5 Long names: Ahmed, Ayesha, Usman

■ LINQ with Classes (Real-World Style)

```
using System;
using System.Collections.Generic;
using System.Linq;

class Student
{
    public int ID { get; set; }
    public string Name { get; set; }
    public float Marks { get; set; }
    public string Department { get; set; }
}

class Program
{
    static void Main()
    {
        var students = new List<Student>
```


■ Interview Questions

A: LINQ is Language Integrated Query — lets you query any data source in C#. Advantages: compile-time error checking, IntelliSense support, readable SQL-like syntax, and works on Collections, DB, XML.

A: Query syntax uses from/where/select keywords like SQL. Method syntax uses lambda expressions (.Where(), .Select()). Both compile to same IL code — choose whichever is more readable.

A: `Select()` is a projection operator. It transforms each element — like SQL's `SELECT` column. Example: `numbers.Select(n => n*2)` doubles every number.

A: LINQ queries are NOT executed when defined — they execute only when iterated (foreach, ToList(), Count() etc.). This is called deferred/lazy execution.

Lecture 20 — Logical & Visual Tree, Dependency Properties

■ Dependency Properties — Definition

A **Dependency Property** is a special property in WPF whose value can be set by multiple sources: animation, data binding, styles, or visual tree inheritance. Unlike regular CLR properties, dependency properties are stored in a centralized dictionary provided by `DependencyObject` — not in individual object fields.

■ CLR Property vs Dependency Property

Feature	CLR Property	Dependency Property
Storage	Stored in object field	Stored in <code>DependencyObject</code> dictionary
Change Notification	Manual (<code>INotifyPropertyChanged</code>)	Built-in
Data Binding	Needs extra code	Natively supported
Animation	Not supported	Built-in support
Inheritance	Not inherited	Can inherit from parent
Default Value	Set in constructor	Set in metadata

■ Defining a Custom Dependency Property

```
using System.Windows;
using System.Windows.Controls;

public class CustomButton : Button
{
    // ■■ Step 1: Register the Dependency Property ■■■■■■■■■■
    public static readonly DependencyProperty CornerRadiusProperty =
        DependencyProperty.Register(
            "CornerRadius",           // property name (string)
            typeof(double),          // property type
            typeof(CustomButton),     // owner type
            new PropertyMetadata(0.0) // default value
        );

    // ■■ Step 2: CLR wrapper (get/set) ■■■■■■■■■■■■■■■■■■■■■■
    public double CornerRadius
    {
        get { return (double)GetValue(CornerRadiusProperty); }
        set { SetValue(CornerRadiusProperty, value); }
    }
}
```

■ Logical Tree vs Visual Tree

WPF represents the UI as two complementary trees:

- **Logical Tree:** The developer-defined structure of XAML elements. Example: `Window > Grid > Button`. It defines the conceptual hierarchy.
- **Visual Tree:** The full expanded rendering tree including all internal parts of controls. A `Button` in logical tree becomes many parts (`Border`, `ContentPresenter`, etc.) in visual tree.

```

<!-- Logical Tree: -->
<Window>
  <Grid>
    <Button Content="Click Me"/>
  </Grid>
</Window>

<!-- Visual Tree (expanded by WPF): -->
Window
  ■■■ Grid
    ■ ■■■ Button
      ■ ■■■ Border
        ■ ■■■ ContentPresenter
          ■ ■■■ TextBlock ("Click Me")

```

■ Routed Events — Bubbling & Tunneling

WPF events can travel up (bubble) or down (tunnel) the visual tree:

- **Bubbling:** Event starts at source and travels UP to root. Example: `MouseLeftButtonDown`.
- **Tunneling:** Event starts at root and travels DOWN to source. Example: `PreviewMouseLeftButtonDown`.
- **Direct:** Event only fires on the element itself.

```

<!-- Bubbling example in XAML -->
<Grid MouseLeftButtonDown="Grid_Click">           <!-- fires when child clicked too -->
  <Button MouseLeftButtonDown="Btn_Click" Content="Click"/>
</Grid>

// In C# – handle event and stop bubbling:
private void Btn_Click(object sender, MouseButtonEventArgs e)
{
    Console.WriteLine("Button clicked");
    e.Handled = true; // stops event from bubbling further up
}

```

■ Interview Questions

Q: What is a Dependency Property?

A: A property that uses WPF's property system. Its value can come from data binding, animation, styles, or inheritance. Stored in `DependencyObject` dictionary, not a regular field.

Q: What is the difference between Logical Tree and Visual Tree?

A: Logical Tree: developer-defined hierarchy from XAML. Visual Tree: full rendering tree including all internal parts of controls generated by WPF.

Q: What is Event Bubbling in WPF?

A: When an event fires on a child element, it travels UP the visual tree, triggering the same event on parent elements. Can be stopped with `e.Handled = true`.

Lecture 24 / 25 — Introduction to ASP.NET MVC

■ What is ASP.NET MVC?

ASP.NET MVC is a web framework by Microsoft for building web applications using the **Model-View-Controller** design pattern. It separates the application into three components: Model (data), View (UI), Controller (logic). It replaced classic ASP and Web Forms.

■ Evolution: ASP → ASP.NET → ASP.NET Core

Version	Released	Extension	Language	Notes
Classic ASP	1998	.asp	VBScript	Server-side scripts in HTML
ASP.NET	2002	.aspx	C#, VB.NET	Built on .NET Framework
ASP.NET MVC	2009	.cshtml	C#	MVC pattern, Razor views
ASP.NET Core	2016	.cshtml	C#	Cross-platform (Win/Linux/Mac)

■ MVC Pattern Explained

- **Model** — C# classes representing data (Student, Product etc.). Contains business logic.
- **View** — .cshtml files (Razor) — HTML + C# for the UI displayed to users.
- **Controller** — C# classes that handle HTTP requests, call Models, return Views.

■ URL Routing in MVC

```
// RouteConfig.cs - Default Route Pattern
routes.MapRoute(
    name: "Default",
    url: "{controller}/{action}/{id}",
    defaults: new { controller = "Home", action = "Index", id = UrlParameter.Optional }
);

// URL Examples:
// /Home/Index          → HomeController.Index()
// /Student/Details/5   → StudentController.Details(5)
// /                    → HomeController.Index() (default)
```

■ Full MVC Example — Student CRUD

```
// ■■ 1. MODEL (Models/Student.cs) ██████████  
namespace SchoolApp.Models  
{  
  
    public class Student  
    {  
        public int ID { get; set; }  
        public string Name { get; set; }  
        public string Email { get; set; }  
        public int Marks { get; set; }  
    }  
}  
  
// ■■ 2. CONTROLLER (Controllers/StudentController.cs) ████████  
using System.Collections.Generic;
```



```

<table border="1" cellpadding="6">
    <tr>
        <th>ID</th><th>Name</th><th>Email</th><th>Marks</th>
    </tr>
    @foreach (var s in Model)
    {
        <tr>
            <td>@s.ID</td>
            <td>@s.Name</td>
            <td>@s.Email</td>
            <td>@s.Marks</td>
            <td><a href="/Student/Details/@s.ID">Details</a></td>
        </tr>
    }
</table>

```

■ MVC Request Flow

- 1. Browser sends request → URL: /Student/Index
- 2. Routing engine matches it to StudentController.Index()
- 3. Controller fetches data from Model (DB/list)
- 4. Controller returns View(data)
- 5. Razor View renders HTML using the data
- 6. HTML response sent back to browser

■ Interview Questions

Q: What is MVC pattern?

A: Model-View-Controller. Model: data and business logic. View: user interface (HTML). Controller: handles requests, connects Model and View. Promotes separation of concerns.

Q: What is ActionResult in MVC?

A: ActionResult is the return type of controller actions. It can be View() (HTML page), RedirectToAction() (redirect), Json() (JSON data), Content() (plain text), etc.

Q: What is the difference between GET and POST in MVC?

A: GET: fetches data, used for displaying pages. POST: sends data to server, used for form submissions. Use [HttpPost] attribute to mark POST actions.

Q: What is ModelState.IsValid?

A: It validates the incoming model data against Data Annotation attributes (like [Required], [Range]). If validation fails, IsValid is false and errors are shown in view.

Lecture 26 — Razor View Engine in ASP.NET MVC

■ Definition

Razor is a view engine in ASP.NET MVC that lets you write C# (server-side) code directly inside HTML. The @ symbol marks the start of C# code. Razor files have .cshtml extension and are compiled on the server before sending HTML to browser.

■ Razor Syntax

```
<!-- Single expression – prefix with @ -->
<p>Today is: @DateTime.Now.ToShortDateString()</p>

<!-- Single line variable declaration -->
@{ var name = "Ahmed"; }

<p>Hello @name!</p>

<!-- Multi-line code block -->
@{
    var course = "Visual Programming";
    var year   = 2025;
    var msg    = $"Welcome to {course} - {year}";
}

<h2>@msg</h2>

<!-- HTML is safe to write inside @{ } block -->
@{
    string color = "red";
}

<p style="color:@color">Colored Text</p>
```

■ Conditions and Loops in Razor

```
@model List<string>  
  
<!-- █ IF-ELSE ████████████████████████████████████████████████████████████████████ -->  
@{  
    int marks = 75;  
}  
@if (marks >= 60)  
{  
    <p style="color:green">PASS ✓</p>  
}  
else  
{  
    <p style="color:red">FAIL ✗</p>  
}  
  
<!-- █ FOR LOOP ████████████████████████████████████████████████████████████████████ -->  
<ul>  
@for (int i = 1; i <= 5; i++)  
{  
    <li>Item @i</li>  
}
```


[illegible]

■ Razor Layouts (_Layout.cshtml)

```
<!-- Shared/_Layout.cshtml - Master page -->
<!DOCTYPE html>
<html>
<head>
    <title>@ViewBag.Title</title>    <!-- dynamic title from view -->
</head>
<body>
    <header><h1>My Website</h1></header>
    <nav>
        <a href="/Home/Index">Home</a> |
        <a href="/Student/Index">Students</a>
    </nav>
    <main>
        @RenderBody()    <!-- child view content goes here -->
    </main>
    <footer>&copy; 2025 COMSATS</footer>
</body>
</html>
```

```
<!-- Views/Home/Index.cshtml - Child view using layout -->
@{
    Layout = "~/Views/Shared/_Layout.cshtml";
    ViewBag.Title = "Home Page";
}

<h2>Welcome to the Home Page</h2>

<p>This content appears inside _Layout.cshtml's @RenderBody()</p>
```

■ HTML Helpers

```
<!-- HTML Helper methods make form elements easier to create -->
@using (Html.BeginForm("Create", "Student", FormMethod.Post))
{
    <div>
        @Html.LabelFor(m => m.Name)
        @Html.TextBoxFor(m => m.Name, new { @class = "form-control" })
        @Html.ValidationMessageFor(m => m.Name)
    </div>
    <div>
        @Html.LabelFor(m => m.Marks)
```

```
@Html.TextBoxFor(m => m.Marks)
</div>
<button type="submit">Save</button>
}
```

■ Interview Questions

Q: What is the Razor View Engine?

A: Razor is the default view engine for ASP.NET MVC. It uses @ to embed C# code inside HTML. Files have .cshtml extension and are compiled server-side before sending to browser.

Q: What is @RenderBody() in Layout?

A: @RenderBody() is a placeholder in _Layout.cshtml that gets replaced by the content of the child view. It's like a template slot.

Q: What is ViewBag?

A: ViewBag is a dynamic property used to pass data from Controller to View. Example: ViewBag.Title = 'Home'; in controller, then @ViewBag.Title in view.

Q: What is the difference between Html.TextBox() and Html.TextBoxFor()?

A: TextBox(name): creates input by string name. TextBoxFor(m => m.Property): strongly typed, uses lambda expression, provides IntelliSense and compile-time checking.

Lecture 27 — LINQ to SQL — ORM with SQL Server

■ Definition

LINQ to SQL is an ORM (Object-Relational Mapping) technology in .NET 3.5+ that maps database tables to C# classes. You write LINQ queries in C# — LINQ to SQL automatically translates them to SQL and executes them. No manual SQL strings needed!

■ Problem with Raw ADO.NET

```
// BAD – raw ADO.NET (no compile-time checking, SQL in strings)
string sql = "SELECT * FROM Students WHERE Marks > @marks";
// Problems:
// 1. SQL is a string – no IntelliSense, typos caught only at runtime
// 2. Results are loosely typed (GetString(0) etc.)
// 3. No compile-time checking
```

■ LINQ to SQL Solution

```
// GOOD - LINQ to SQL (type-safe, IntelliSense, no SQL strings)
// After mapping DB table to C# class, query like this:
using (SchoolDataContext db = new SchoolDataContext())
{
    // This C# LINQ query auto-converts to SQL:
    // SELECT * FROM Students WHERE Marks > 60 ORDER BY Name
    var topStudents = from s in db.Students
                      where s.Marks > 60
                      orderby s.Name
                      select s;

    foreach (var s in topStudents)
        Console.WriteLine($"{s.Name}: {s.Marks}");
}
```

■ DataContext — Full CRUD Example

[illegible]

```
static void GetPassStudents()
{
    var passed = db.Students.Where(s => s.Marks >= 50);
    foreach (var s in passed)
        Console.WriteLine(s.Name);
}

// ■■ INSERT ████████████████████████████████████████████████████████████
static void AddStudent()
{
    Student newStudent = new Student
    {
        Name = "Ahmed Khan",
        Email = "ahmed@comsats.pk",
        Marks = 78
    };
    db.Students.InsertOnSubmit(newStudent); // queue insert
    db.SubmitChanges();                     // execute SQL INSERT
    Console.WriteLine("Student added!");
}

// ■■ UPDATE ████████████████████████████████████████████████████████████
static void UpdateMarks(int id, int newMarks)
{
    var student = db.Students.SingleOrDefault(s => s.ID == id);
    if (student != null)
    {
        student.Marks = newMarks; // just change the property
        db.SubmitChanges();       // auto-generates UPDATE SQL
        Console.WriteLine("Updated!");
    }
}

// ■■ DELETE ████████████████████████████████████████████████████████████
static void DeleteStudent(int id)
{
    var student = db.Students.SingleOrDefault(s => s.ID == id);
    if (student != null)
    {
        db.Students.DeleteOnSubmit(student); // queue delete
        db.SubmitChanges();                   // execute DELETE SQL
    }
}

static void Main()
{
    GetAllStudents();
    AddStudent();
    UpdateMarks(1, 90);
    DeleteStudent(5);
}
```

```
}
```

■ LINQ to SQL vs ADO.NET

Feature	ADO.NET	LINQ to SQL
Query style	Raw SQL strings	C# LINQ queries
Type safety	Loosely typed	Strongly typed
IntelliSense	No (for SQL)	Yes
Compile checking	No	Yes
Code amount	More boilerplate	Much less code
DB Support	All via providers	SQL Server only
Performance	Slightly faster	Slight overhead

■ Interview Questions

Q: What is ORM?

A: Object-Relational Mapping. It maps database tables to C# classes and handles SQL generation automatically. LINQ to SQL and Entity Framework are both ORMs.

Q: What does SubmitChanges() do?

A: It sends all pending changes (inserts, updates, deletes) to the database as a single transaction. Without it, no DB changes happen.

Q: What is the difference between LINQ to SQL and Entity Framework?

A: LINQ to SQL: supports SQL Server only, simpler, 1:1 table-to-class mapping. Entity Framework: supports multiple databases, more complex mappings, supports Code First/Database First approaches.

Lecture 29 / 30 — Entity Framework (EF) — Advanced ORM

■ Definition

Entity Framework is Microsoft's official ORM for .NET. It maps .NET classes to database tables and translates LINQ queries to SQL. EF supports multiple databases (SQL Server, Oracle, MySQL etc.) and offers three development approaches: Database First, Model First, and Code First.

■ EF Approaches

- **Database First:** DB already exists → EF generates C# classes from it.
- **Model First:** Design entities visually in designer → EF creates DB.
- **Code First:** Write C# classes first → EF creates/updates DB automatically. (Most popular today)

■ Code First — Full Example

[illegible]

[illegible]

■ EF vs LINQ to SQL vs ADO.NET

Feature	ADO.NET	LINQ to SQL	Entity Framework
DB Support	All	SQL Server only	All (via providers)
Mapping	Manual	1:1 table:class	Complex mappings

Code First	No	No	Yes
Multiple tables	Manual joins	Limited	Navigation props
Migration support	No	No	Yes

■ Interview Questions

Q: What is DbContext?

A: DbContext is the main class in Entity Framework. It represents a session with the database. It contains DbSet properties for each entity/table. SaveChanges() pushes all pending changes to DB.

Q: What is Code First approach?

A: You write C# POCO (Plain Old C# Object) classes first. Entity Framework automatically creates the database schema from them. You can use Migrations to update DB when classes change.

Q: What is a Navigation Property?

A: A property that represents a relationship between entities. Example: Student has ICollection — lets you access related enrollments without writing JOIN queries.

Q: What does SaveChanges() do in EF?

A: It wraps all pending INSERT/UPDATE/DELETE operations in a transaction and sends them to the database as SQL. Without it, changes only exist in memory.

Lecture 31 — Model Binding in ASP.NET MVC

■ Definition

Model Binding is an automatic process in ASP.NET MVC that maps HTTP request data (form fields, query strings, route values) to C# action method parameters or model classes. It eliminates the need to manually read `Request.Form["fieldName"]` for every field.

■ Types of Model Binding

- **No Binding:** Action method has no parameters — reads nothing from request.
- **Simple Binding:** Primitive parameters (int id, string name) matched by name.
- **Class Binding:** Entire model class passed as parameter — all properties auto-filled.
- **Complex Binding:** Multiple related models bound in one view.
- **FormCollection Binding:** FormCollection object captures all form values.
- **[Bind] Attribute:** Restrict which properties are bound (security).

■ Full Example — All Binding Types

[illegible]

[illegible]

```

<div>
    @Html.LabelFor(m => m.Name)
    @Html.TextBoxFor(m => m.Name, new { @class = "form-control" })
    @Html.ValidationMessageFor(m => m.Name, "", new { @class = "text-danger" })
</div>

<div>
    @Html.LabelFor(m => m.Email)
    @Html.TextBoxFor(m => m.Email)
    @Html.ValidationMessageFor(m => m.Email)
</div>

<div>
    @Html.LabelFor(m => m.Marks)
    @Html.TextBoxFor(m => m.Marks)
    @Html.ValidationMessageFor(m => m.Marks)
</div>

<button type="submit" class="btn btn-primary">Save Student</button>
}

@Html.ValidationSummary(true, "Please fix errors:")

```

■ Data Annotations for Validation

```

// Common Data Annotations used in Models:
public class Student
{
    [Required]                                // field cannot be empty
    [StringLength(50, MinimumLength=2)]       // length 2 to 50
    public string Name { get; set; }

    [EmailAddress]                            // must be valid email
    public string Email { get; set; }

    [Range(1, 100)]                           // must be between 1 and 100
    public int Marks { get; set; }

    [RegularExpression(@"^\d{11}$")]           // exactly 11 digits
    public string Phone { get; set; }

    [DataType(DataType.Date)]                 // date picker in UI
    public DateTime DOB { get; set; }
}

```

■ Interview Questions

Q: What is Model Binding in ASP.NET MVC?

A: Model Binding automatically maps HTTP request data (form fields, query strings) to C# action parameters or model objects. MVC's DefaultModelBinder handles this behind the scenes.

Q: What is the [Bind(Include)] attribute used for?

A: It's used to whitelist only specific properties for binding. This prevents over-posting/mass assignment attacks where a hacker sends extra fields in the request.

Q: What are Data Annotations?

A: Attributes placed on model properties to define validation rules. Examples: [Required], [Range], [EmailAddress], [StringLength]. ModelState.IsValid checks these automatically.

Q: What is the difference between TempData, ViewBag, and ViewData?

A: ViewData/ViewBag: pass data from Controller to View for ONE request. TempData: persists for TWO requests (survives redirect). Use TempData for redirect success messages.

Q: What is [HttpPost] attribute?

A: Restricts an action method to only handle HTTP POST requests. Without it, the action accepts all request types. Use [HttpGet] or [HttpPost] to differentiate between displaying form and processing it.

End of Notes

Visual Programming — After Mid Complete Notes

COMSATS University Islamabad, Attock Campus

Topics Covered: Threads · ADO.NET · async/await · WPF Data Binding

LINQ · Dependency Properties · ASP.NET MVC · Razor · LINQ to SQL · EF ·
Model Binding